FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

3

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Student Name	Gabriel A. Cuesta
Section:	AN43
Professor:	Crisola G. Tan
Date:	10-25-2026

I. BACKGROUND INFORMATION

Case-Based Reasoning (CBR) is an intelligent problem-solving approach used in **Knowledge Representation and Reasoning (KRR)**. Instead of relying solely on predefined rules, CBR systems solve new problems by referring to past experiences, also known as cases. These cases contain previous problems and their successful solutions. When a new situation arises, the system identifies the most similar past case, adapts the old solution to fit the new context, and applies it. Afterward, the new case is saved, allowing the system to learn and continuously improve.

CBR is widely used in various fields, such as disaster response, legal assistance, and intelligent tutoring, because it mimics how humans naturally make decisions. For example, emergency responders rely on past typhoon data to plan evacuations; lawyers examine previous legal cases to build arguments; and tutoring systems look at prior student mistakes to provide customized guidance. In all these examples, past knowledge becomes the foundation for solving future problems more effectively.

This activity allows students to experience how CBR works by developing a simple, functional CBR program based on real-world scenarios. Through pair programming, students will learn how to represent cases, implement similarity assessment, adapt solutions, and store new learning experiences—reflecting the complete CBR cycle. This hands-on approach strengthens understanding of how intelligent systems use experiential knowledge to support reasoning, decision-making, and learning.

II. TECHNICAL ASSESSMENT

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Objective

At the end of this laboratory activity, students are expected to:

1. **Understand and apply the Case-Based Reasoning (CBR) cycle**, including retrieval, reuse, revision, and retention.
2. **Represent cases effectively** using appropriate data structures such as dictionaries, arrays, JSON objects, or classes.
3. **Implement a similarity assessment method** to identify relevant past cases based on defined attributes.
4. **Adapt retrieved solutions** to meet the needs of a new problem scenario based on the chosen case study.
5. **Demonstrate learning in CBR** by saving the newly solved case into the system's case base.
6. **Develop collaboration skills** through pair programming and joint problem analysis.
7. **Apply knowledge representation and reasoning concepts** to real-world problems in disaster response, legal reasoning, or intelligent tutoring.

Materials Needed

1. Hardware
    - A laptop or desktop computer
    - Stable internet connection (for research, documentation, submission)
2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

4. Reference Materials

Lecture notes on:

- Knowledge Representation & Reasoning
- Case-Based Reasoning cycle
- Similarity measures

Provided case studies (Disaster DSS, Legal DSS, Intelligent Tutoring System)

5. Pair Collaboration Tools (optional but useful)
    - Google Drive for shared documentation
    - GitHub/GitLab for code collaboration

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Please ensure that you are grouped as pairs in the People tab.
3. Select only one case study from the list below.
4. Include clear comments in your code to describe its logic.
5. Each program should display the correct input, process, and output.
6. Ensure that your script displays clear and readable outputs.
7. Compile all Python codes and corresponding outputs into one PDF file.
8. Save it as **TA3_lastname1_lastname2.pdf**
   (Example: TA2_Patilano_Loverez.pdf)
9. Submit the **python file** and **PDF** file.

📌 Step-by-Step Instructions

1. Form a pair

Choose a partner. Each pair will work collaboratively on research, coding, debugging, and presenting your results.

2. Choose ONE (1) case study

Select only one case study from the list below. Your program must be based on the scenario and logic of the chosen system:

Case Study Options:

1. *Disaster Response Decision Support System (DSS)*
2. *AI-Based Legal Decision Support System*
3. *Intelligent Tutoring System (ITS)*

3. Define your case base

Create at least **3–5 past cases** stored as dictionaries, JSON objects, arrays, or any structure.

Example structure:

```
{
  "problem": {...},
  "solution": {...}
}
```

4. Implement similarity assessment

Your program must include a way to measure similarity between the new case and past cases. Examples:

- Attribute matching
- Weighted scoring
- Simple rule-based comparison

Output should show: (sample)

✓ Similarity score or ranking

✓ The most relevant past case retrieved

5. Adapt the solution

Modify the retrieved solution so it fits the new problem scenario.
This part should reflect the logic of your chosen case study:

Examples:

- **DSS:** Adjust evacuation plans based on population size
- **Legal DSS:** Modify arguments due to new digital evidence
- **ITS:** Give personalized feedback based on student errors

6. Demonstrate learning (retain stage)

After generating a final solution, store the new case into your case base.

Your program must show:

✓ "New case saved" or similar confirmation

✓ Updated case list

7. Display clear program outputs

Your output must include:

- New problem description

- Retrieved similar case
- Adapted final solution
- Updated case base

8. Submission Requirements

Submit the following:

A. Program File

- Python

B. Short Documentation (1–2 pages)

Include:

- Title of chosen case study
- Description of your case base
- Explanation of similarity method
- Adaptation logic
- Screenshots of program execution

C. Pair Reflection (5–7 sentences)

Discuss:

- What each partner contributed
- What challenges you encountered
- What you learned about CBR

Case Studies:

Case Study 1: Disaster Response Decision Support System

A CBR-based DSS helps local government units respond to natural disasters.

- Past cases include typhoons, floods, and earthquakes.
- New event: *Typhoon with expected flooding in urban areas.*
- System retrieves similar events: *Typhoon Ursula (2019)* and *Typhoon Odette (2021)*.
- It adapts recommended responses such as evacuation strategy, resource allocation, and shelter capacity.
- After execution, the system stores new outcomes for improved future reasoning.

Learning Outcome:

CBR helps governments respond faster by using *experiential knowledge* rather than rebuilding decisions from scratch.

Case Study 2: AI-Based Legal Decision Support System

A legal CBR tool assists lawyers by retrieving previous court cases similar to the current legal issue.

Scenario:

Client is charged with "theft involving electronic transfer." System retrieves past cases involving:

- online theft
- digital fraud
- unauthorized bank transfers

Adaptation:

The reasoning engine adjusts argument patterns to match the new technical context.

Learning:
After final verdict, the case is stored—enhancing the system's knowledge for future cybercrime-related offenses.

Case Study 3: Intelligent Tutoring System

A CBR system identifies the learning struggles of students.
Student submits a math solution with repeated errors.
System retrieves past students with similar error patterns (e.g., misusing formulas).
It adapts feedback and personalized lessons based on successful interventions.
Learning:
System improves by retaining new student profiles and outcomes.

Your Code - Laboratory Tasks ⇩

Code

```
import json
import os

FILE_NAME = "disaster_cases.json"

# 1. INITIAL SAMPLE CASES

sample_cases = [
    {
        "case_id": 1,
        "name": "Typhoon Ursula (2019) - Sample",
        "disaster_type": "Typhoon",
        "flood_risk": "High",
        "area_type": "Coastal",
        "population": 10000,
        "solution": {
            "evacuation": "Evacuate residents in high-risk areas",
            "resources": "Prepare food, water, and rescue boats",
            "shelter": 2000
        }
    },
    {
        "case_id": 2,
        "name": "Typhoon Odette (2021) - Sample",
        "disaster_type": "Typhoon",
        "flood_risk": "Medium",
        "area_type": "Coastal",
```

```
        "population": 15000,
        "solution": {
            "evacuation": "Conduct preemptive evacuation",
            "resources": "Prepare food, water, and medical supplies",
            "shelter": 3000
        }
    },
    {
        "case_id": 3,
        "name": "Urban Flooding - Sample",
        "disaster_type": "Flood",
        "flood_risk": "High",
        "area_type": "Urban",
        "population": 8000,
        "solution": {
            "evacuation": "Move residents to elevated safe zones",
            "resources": "Prepare rescue vehicles and medical teams",
            "shelter": 1600
        }
    },
    {
        "case_id": 4,
        "name": "Earthquake Emergency - Sample",
        "disaster_type": "Earthquake",
        "flood_risk": "Low",
        "area_type": "Urban",
        "population": 12000,
        "solution": {
            "evacuation": "Move residents away from unsafe buildings",
            "resources": "Prepare first aid and rescue teams",
            "shelter": 2400
        }
    },
    {
        "case_id": 5,
        "name": "Rural Flood Emergency - Sample",
        "disaster_type": "Flood",
        "flood_risk": "Medium",
        "area_type": "Rural",
        "population": 5000,
        "solution": {
            "evacuation": "Evacuate residents near rivers",
            "resources": "Prepare boats, food, and clean water",
```

```
            "shelter": 1000
        }
    }
]


# 2. LOAD OR CREATE THE CASE BASE

def load_cases():
    if os.path.exists(FILE_NAME):
        with open(FILE_NAME, "r") as file:
            return json.load(file)

    # Only used when the JSON file does not exist.
    with open(FILE_NAME, "w") as file:
        json.dump(sample_cases, file, indent=4)

    return sample_cases.copy()


def save_cases(cases):
    with open(FILE_NAME, "w") as file:
        json.dump(cases, file, indent=4)


# 3. GET USER INPUT

def choose_option(question, options):
    print("\n" + question)

    for number, option in enumerate(options, start=1):
        print(f"{number}. {option}")

    while True:
        choice = input("Enter a number or type your choice: ").strip()

        # Allow the user to enter a number.
        if choice.isdigit():
            number = int(choice)

            if 1 <= number <= len(options):
                return options[number - 1]
```

```
        # Also allow the user to type the option.
        for option in options:
            if choice.lower() == option.lower():
                return option

        print("Invalid choice. Please try again.")


def get_population():
    while True:
        value = input(
            "\nEnter the estimated affected population: "
        ).strip()

        try:
            population = int(value)

            if population > 0:
                return population

            print("Population must be greater than zero.")

        except ValueError:
            print("Please enter a valid whole number.")


# 4. CALCULATE SIMILARITY

def calculate_similarity(new, old):
    score = 0

    # Disaster type: 35 points
    if new["disaster_type"] == old["disaster_type"]:
        score += 35

    # Flood risk: 30 points
    if new["flood_risk"] == old["flood_risk"]:
        score += 30

    # Area type: 20 points
    if new["area_type"] == old["area_type"]:
        score += 20
```

```
    # Population similarity: 15 points
    smaller = min(new["population"], old["population"])
    larger = max(new["population"], old["population"])

    score += (smaller / larger) * 15

    return round(score, 2)


# 5. ADAPT THE RETRIEVED SOLUTION

def adapt_solution(new_case, past_case):
    solution = past_case["solution"].copy()

    disaster = new_case["disaster_type"]
    flood_risk = new_case["flood_risk"]
    area = new_case["area_type"]
    population = new_case["population"]

    # Adapt evacuation based on disaster type.
    if disaster == "Typhoon":
        evacuation = (
            "Prepare evacuation routes and move residents "
            "away from typhoon and flooding danger zones"
        )
    elif disaster == "Flood":
        evacuation = (
            "Move residents to elevated, accessible safe zones"
        )
    elif disaster == "Earthquake":
        evacuation = (
            "Move people away from damaged buildings and "
            "assess structures before re-entry"
        )
    elif disaster == "Landslide":
        evacuation = (
            "Move residents away from unstable slopes "
            "and landslide-prone areas"
        )
    elif disaster == "Volcanic Eruption":
        evacuation = (
            "Follow official evacuation orders and move "
            "residents away from volcanic hazard zones"
```

```
        )
    else:
        evacuation = (
            "Follow official emergency instructions and "
            "move residents away from identified hazards"
        )

    # Adjust evacuation intensity for population size.
    if population >= 15000:
        evacuation += (
            ". Organize phased evacuations across multiple "
            "zones and coordinate transport"
        )
    elif population >= 8000:
        evacuation += (
            ". Assign evacuation teams to high-risk areas"
        )
    else:
        evacuation += (
            ". Prioritize the most vulnerable residents"
        )

    # Increase precautions for high flood risk.
    if flood_risk == "High":
        evacuation += (
            ". Prioritize flood-prone communities and "
            "monitor official flood warnings"
        )

    # Adapt resource allocation.
    if population >= 15000:
        resources = (
            "Deploy additional rescue teams, transport, "
            "food, drinking water, medical supplies, "
            "and emergency personnel"
        )
    elif population >= 8000:
        resources = (
            "Prepare rescue teams, food, drinking water, "
            "medical supplies, and transport"
        )
    else:
        resources = (
```

```
            "Prepare essential food, water, first aid, "
            "and local emergency personnel"
        )

    if area == "Coastal":
        resources += "; prepare appropriate coastal rescue equipment"
    elif area == "Urban":
        resources += "; coordinate traffic control and accessible routes"
    elif area == "Mountainous":
        resources += "; assess road access and isolated communities"
    else:
        resources += "; identify remote communities and available transport"

    shelter_capacity = int(population * 0.20)

    # Return the adapted solution.
    return {
        "evacuation": evacuation,
        "resources": resources,
        "shelter": shelter_capacity
    }


# 6. MAIN PROGRAM

def main():
    case_base = load_cases()

    print("\n" + "=" * 50)
    print("DISASTER RESPONSE DECISION SUPPORT SYSTEM")
    print("=" * 50)

    print(f"\nExisting cases in database: {len(case_base)}")

    # Collect a new disaster scenario.
    disaster_type = choose_option(
        "Choose the disaster type:",
        [
            "Typhoon",
            "Flood",
            "Earthquake",
            "Landslide",
            "Volcanic Eruption",
```

```
        "Other"
    ]
)

flood_risk = choose_option(
    "Choose the flood risk:",
    ["Low", "Medium", "High"]
)

area_type = choose_option(
    "Choose the affected area type:",
    ["Urban", "Rural", "Coastal", "Mountainous"]
)

population = get_population()

new_case = {
    "case_id": max(
        [case.get("case_id", 0) for case in case_base],
        default=0
    ) + 1,
    "name": f"New {disaster_type} Case",
    "disaster_type": disaster_type,
    "flood_risk": flood_risk,
    "area_type": area_type,
    "population": population
}

# Display the new problem.
print("\n" + "-" * 50)
print("NEW PROBLEM DESCRIPTION")
print("-" * 50)
print("Disaster type:", disaster_type)
print("Flood risk:", flood_risk)
print("Area type:", area_type)
print("Affected population:", population)

# Calculate scores for all past cases.
ranked_cases = []

for case in case_base:
    score = calculate_similarity(new_case, case)
    ranked_cases.append((score, case))
```

```
ranked_cases.sort(
    key=lambda item: item[0],
    reverse=True
)

# Display similarity rankings.
print("\nSIMILARITY RANKINGS")

for rank, (score, case) in enumerate(
    ranked_cases, start=1
):
    print(
        f"{rank}. {case['name']} - {score}%"
    )

# Retrieve the highest-scoring case.
best_score, best_case = ranked_cases[0]

print("\n" + "-" * 50)
print("RETRIEVED PAST CASE")
print("-" * 50)
print("Case:", best_case["name"])
print("Similarity score:", best_score, "%")
print("Original evacuation:", best_case["solution"]["evacuation"])
print("Original resources:", best_case["solution"]["resources"])

# Adapt the response.
final_solution = adapt_solution(new_case, best_case)

print("\n" + "-" * 50)
print("ADAPTED FINAL SOLUTION")
print("-" * 50)
print("Evacuation plan:", final_solution["evacuation"])
print("Resource allocation:", final_solution["resources"])
print(
    "Target shelter capacity:",
    final_solution["shelter"],
    "people"
)

# Retain the new case and its solution.
new_case["solution"] = final_solution
```

```
    case_base.append(new_case)

    save_cases(case_base)

    print("\nNEW CASE SAVED SUCCESSFULLY!")
    print("File:", FILE_NAME)

    # Show the updated case base.
    print("\n" + "-" * 50)
    print("UPDATED CASE BASE")
    print("-" * 50)
    print("Total cases:", len(case_base))

    for case in case_base:
        print(
            f"Case {case['case_id']}: {case['name']} | "
            f"Type: {case['disaster_type']} | "
            f"Risk: {case['flood_risk']} | "
            f"Area: {case['area_type']} | "
            f"Population: {case['population']}"
        )

    print("\nCBR PROCESS COMPLETED.")


# Run the program.
if __name__ == "__main__":
    main()
```